Control Statements (If – Else & Switch case)

If-Else Statements:

1. If Statement

Used to execute a block of code **only if the condition is true**.

Example:

```
int age = 20;
if (age >= 18) {
    printf("You are eligible to vote.");
}
```

Output:

```
You are eligible to vote.
```

2. If – Else Statement

Executes one block if condition is true, otherwise executes another block.

Syntax:

```
if (condition) {
    // code if true
} else {
    // code if false
}
```

Example:

```
int marks = 35;
if (marks >= 40) {
    printf("Pass");
} else {
    printf("Fail");
}
```

Output:

```
Fail
```

3. If – Else If – Else Ladder

Used when we have **multiple conditions**.
Conditions are checked **top to bottom**, and the first true condition executes.

Example:

```
int marks = 75;

if (marks >= 90) {
    printf("Grade A");
} else if (marks >= 75) {
    printf("Grade B");
} else if (marks >= 50) {
    printf("Grade C");
} else {
```

```
    printf("Fail");
}
```

Output:

```
Grade B
```

4. Nested If Statement

An `if` statement **inside another if**.

Example:

```
int age = 20;
int citizen = 1;  // 1 = Indian, 0 = Other

if (age >= 18) {
    if (citizen == 1) {
        printf("Eligible for voting in India.");
    } else {
        printf("Not an Indian citizen.");
    }
} else {
    printf("Not eligible (age below 18).");
}
```

Output:

```
Eligible for voting in India.
```

Switch Case Statement

- The **switch statement** is used when we need to execute **one block of code out of many choices**.
- It works with **int, char, enum** types (not with float/double or strings).
- It is an **alternative to multiple if–else if** statements.

Important Points

1. `expression` is evaluated **once**.
2. The value is compared with each `case`.
3. If a match is found → that block runs.
4. `break;` stops execution and exits the switch.
5. If no case matches → `default` runs (optional).

Example 1: Simple Switch

```
#include <stdio.h>
int main() {
  int day = 3;

  switch(day) {
    case 1:
      printf("Monday");
      break;
    case 2:
      printf("Tuesday");
      break;
    case 3:
      printf("Wednesday");
```

```
    break;
   default:
    printf("Invalid Day");
  }
  return 0;
}
```

Output:

```
Wednesday
```

Restrictions on Switch Values in C

The `switch` statement in C has some restrictions on the values used in it:

1. The `switch` expression must be of an **integer or character type** such as `int`, `char`, `short`, or `long`.
2. **Floating-point values** like `float` and `double` cannot be used directly with `switch`.
3. The value of each `case` must be a **constant integer or character value**.
4. **Duplicate case values** are not allowed.
5. A **range of values** cannot be directly specified in a `case`.

Break and Default with Switch in C

1. `break` with Switch

The **`break` statement** is used to stop the execution of the `switch` statement.

When a matching `case` is found, its statements are executed. Without `break`, the program continues executing the statements of the next cases. This is called **fall-through**.

Example:

```
int choice = 2;

switch(choice)
{
    case 1:
        printf("One");
        break;

    case 2:
        printf("Two");
        break;

    case 3:
        printf("Three");
        break;
}
```

Here, when `choice` is `2`, **"Two"** is printed and `break` terminates the switch.

2. default with Switch

The **`default` case** is executed when none of the `case` values match the `switch` expression.

It is similar to the **`else` statement** in an `if-else` structure.

Example:

```
int choice = 5;

switch(choice)
{
    case 1:
        printf("One");
        break;

    case 2:
        printf("Two");
        break;

    default:
        printf("Invalid choice");
}
```

Since `5` does not match any case, the `default` block is executed.

Note: The `default` case is optional.

Comparison Between if-else and switch-case

if-else	switch-case
Used for **conditions and decision making**.	Used for selecting one option from **multiple fixed values**.
Can check **ranges and complex conditions**.	Mainly checks **constant values**.
Supports operators like `>`, `<`, `>=`, `<=`, `&&`, `	Supports Only Cases
Can work with different types of conditions.	The switch expression must be of an **integral type** such as `int` or `char`.
Suitable for **complex conditions**.	Suitable for **menu-driven programs and fixed choices**.
Uses `else if` for multiple conditions.	Uses multiple `case` statements.
`else` is used when no condition is true.	`default` is used when no case matches.

Example

Using if-else:

```
if(choice == 1)
    printf("Add");
else if(choice == 2)
    printf("Delete");
else
    printf("Invalid choice");
```

Using switch-case:

```
switch(choice)
{
    case 1:
        printf("Add");
        break;

    case 2:
```

```
        printf("Delete");
        break;

    default:
        printf("Invalid choice");
}
```

Key Difference

Use if-else when you have conditions, comparisons, or ranges.
Use switch-case when you need to compare one expression with multiple fixed values.

Loops and Iteration in C

A **loop** is a control structure that repeatedly executes a statement or group of statements as long as a specified condition permits the repetition.

The process of repeatedly executing a set of statements is called **iteration**.

For example, if we want to print numbers from 1 to 100, writing 100 separate `printf()` statements would be inefficient. A loop can perform the same task with only a few statements.

Definition: A loop is a control structure used to execute a block of statements repeatedly until a given condition becomes false or another terminating condition occurs.

Need for Loops

Loops are useful because they:

- Reduce repetitive code.
- Make programs shorter and easier to understand.
- Save programming effort.
- Make repetitive calculations easier.
- Allow processing of collections of data.
- Improve program organization.

Types of Loops in C

C mainly provides three looping statements:

1. **`for` loop**
2. **`while` loop**
3. **`do-while` loop**

They can broadly be classified as:

Entry-controlled loops: `for`, `while`
Exit-controlled loop: `do-while`

1. The for Loop

A **`for` loop** is commonly used when the initialization, test condition, and update can be conveniently expressed together, especially when the number or pattern of iterations is known.

Syntax

```
for (initialization; condition; update)
{
    statements;
}
```

It has three main control expressions:

Initialization → Condition → Body → Update → Condition again

Example: Print Numbers 1 to 5

```
#include <stdio.h>

int main(void)
{
    int i;
    for (i = 1; i <= 5; i++)
    {
        printf("%d ", i);
    }
    return 0;
}
```

Output:

```
1 2 3 4 5
```

Working of `for` Loop

```
for (i = 1; i <= 5; i++)
```

`i = 1` → initializes the loop variable.

`i <= 5` → checks whether the loop should continue.

`i++` → increases `i` by 1 after each iteration.

The initialization is normally performed only once. The condition is tested before each iteration.

Flow of for Loop

Initialization → Test Condition → Loop Body → Update → Test Condition → ... → Exit

Since the condition is tested before execution of the body, a `for` loop may execute **zero times**.

2. The while Loop

A **`while` loop** repeatedly executes a block of statements while its controlling expression evaluates as true (nonzero).

It is particularly useful when the number of iterations is not known in advance and repetition depends on a condition.

Syntax

```
while (condition)
{
    statements;
}
```

Example

```
#include <stdio.h>

int main(void)
{
    int i = 1;
    while (i <= 5)
    {
        printf("%d ", i);
        i++;
    }
    return 0;
}
```

Output:

```
1 2 3 4 5
```

Working

Initially:

```
i = 1;
```

Then the condition:

`i <= 5` is checked.

If it is true, the body executes. After `i++`, the condition is tested again.

When `i` becomes `6`, the condition becomes false and the loop terminates.

Important Point

`while` is an **entry-controlled loop**, because its condition is checked before the body is executed.

Therefore, the body may execute **zero times**.

3. The do-while Loop

The **`do-while` loop** is an exit-controlled loop.

Unlike `for` and `while`, its condition is checked **after** executing the loop body.

Therefore, its body executes **at least once**.

Syntax

```
do
{
    statements;
}
```

```
while (condition);
```

Notice the semicolon `;` after the `while(condition)`.

Example

```
#include <stdio.h>

int main(void)
{
    int i = 1;
    do
    {
        printf("%d ", i);
        i++;
    }
    while (i <= 5);
    return 0;
}
```

Output:

```
1 2 3 4 5
```

Why Does `do-while` Execute At Least Once?

```
int i = 10;

do
{
    printf("Hello");
}
while (i < 5);
```

Although `i < 5` is false, `"Hello"` is still printed once because the condition is checked **after** the body.

Difference Between `for`, `while`, and `do-while`

Basis	`for`	`while`	`do-while`
Control	Entry-controlled	Entry-controlled	Exit-controlled
Condition checked	Before body	Before body	After body
Minimum executions	0	0	1
Initialization	Commonly in loop header	Usually before loop	Usually before loop
Update	Commonly in loop header	Usually inside body	Usually inside body
Typical use	Convenient for counter-controlled loops	Convenient for condition-controlled loops	Useful when body must run at least once

Multiple Loop Variables

A `for` loop can control more than one variable. The comma operator can be used in the initialization and update expressions.

Example

```
#include <stdio.h>
```

```
int main(void)
{
    int i, j;
    for (i = 1, j = 5; i <= 5; i++, j--)
    {
        printf("%d %d\n", i, j);
    }
    return 0;
}
```

Output:

```
1 5
2 4
3 3
4 2
5 1
```

Here two loop variables are used:

`i` increases from 1 to 5.

`j` decreases from 5 to 1.

Important Point

```
for (i = 1, j = 5; i <= 5; i++, j--)
```

multiple expressions are used in the initialization and update sections, while the middle

Nested Loops

When one loop is placed inside another loop, it is called a **nested loop**.

A nested loop is a loop that appears within the body of another loop.

The outer loop controls how many times the inner loop as a whole is entered, while the inner loop completes its applicable iterations for each outer iteration.

Syntax

```
for (initialization; condition; update)
{
    for (initialization; condition; update)
    {
        statements;
    }
}
```

Example

```
#include <stdio.h>

int main(void)
{
    int i, j;

    for (i = 1; i <= 3; i++)
    {
        for (j = 1; j <= 3; j++)
```

```
        {
            printf("* ");
        }

        printf("\n");
    }

    return 0;
}
```

Output:

```
* * *
* * *
* * *
```

The outer loop controls the **rows**, while the inner loop prints the symbols in each row.

Uses of Nested Loops

Nested loops are commonly used for:

- Pattern printing
- Tables
- Matrix processing
- Two-dimensional arrays
- Repeated comparisons
- Row-and-column operations

Number of Iterations

If the outer loop runs `m` times and, for every outer iteration, the inner loop runs `n` times, the inner body executes:

m × n times

For example, if both loops execute 5 times:

5 × 5 = 25 iterations

Assignment Operators

An **assignment operator** is used to assign a value to a variable.

The basic assignment operator is: `=`

Example:

```
int a;

a = 10;
```

Here, the value `10` is assigned to `a`.

Compound Assignment Operators

C provides several compound assignment operators.

Operator	Example	Meaning
=	a = 10	Assign value
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5

There are also compound assignment forms for bitwise and shift operators,

such as &=, |=, ^=, <<=, and >>=.

Example

```
int a = 10;

a += 5;
```

Now:

```
a = 15
```

Similarly:

```
a *= 2;
```

If a was 15, it becomes:

```
30
```

Important Difference: = and ==

This is a common examination and programming-error point.

= means **assignment**.

```
a = 10;
```

== means **comparison for equality**.

```
if (a == 10)
```

= assigns a value, whereas == compares two values for equality.

The `break` Statement

The **`break` statement** is used to immediately terminate the nearest enclosing loop or `switch` statement.

Syntax: `break;`

When `break` is executed inside a loop, control moves to the first statement following that loop.

Example

```
#include <stdio.h>
```

```
int main(void)
{
    int i;
    for (i = 1; i <= 10; i++)
    {
        if (i == 5)
        {
            break;
        }
        printf("%d ", i);
    }

    return 0;
}
```

Output:

```
1 2 3 4
```

When `i` becomes 5, `break` terminates the loop.

Uses of `break`

`break` is used to:

- Terminate a loop before its normal completion.
- Exit a `switch` case structure.
- Stop searching when the required item has been found.
- Exit the nearest enclosing loop when a particular condition occurs.

Important Nested-Loop Point

If `break` appears inside the inner loop of nested loops, it terminates **only that nearest inner loop**, not all surrounding loops automatically.

The `continue` Statement

The **`continue` statement** skips the remaining statements of the current loop iteration and proceeds with the loop's next iteration according to the rules of that loop.

Syntax:

```
continue;
```

Example

```
#include <stdio.h>

int main(void)
{
    int i;

    for (i = 1; i <= 5; i++)
    {
        if (i == 3)
        {
            continue;
        }
```

```
        printf("%d ", i);
    }

    return 0;
}
```

Output:

```
1 2 4 5
```

When `i == 3`, `continue` skips the `printf()` for that iteration. The loop itself does **not** terminate.

Difference Between `break` and `continue`

This is an important short-answer question.

`break`	`continue`
Terminates the nearest enclosing loop or `switch`.	Skips the remaining part of the current loop iteration.
Control goes outside the loop/switch.	Control proceeds to the loop's next iteration step/test as appropriate.
Stops further iterations of that loop.	Does not terminate the loop by itself.
Used when repetition should stop completely.	Used when only a particular iteration should be skipped.

Easy Example

Suppose:

```
1 2 3 4 5
```

With `break` at 3:

```
1 2
```

The loop stops.

With `continue` at 3:

```
1 2 4 5
```

Only 3 is skipped.

Infinite Loop

An **infinite loop** is a loop that continues indefinitely because its termination condition never becomes false or no terminating action occurs.

Example:

```
while (1)
{
    printf("Hello\n");
}
```

Another common form is:

```
for (;;)
{
    /* statements */
}
```

Such loops are sometimes intentionally used in programs, but an unintended infinite loop is generally a logical error.

Functions in C

A **function** is a self-contained block of code designed to perform a specific task. Instead of writing the same code repeatedly, a programmer can place that code inside a function and call the function whenever required.

Functions are an important feature of **structured and modular programming** because they divide a large program into smaller and manageable units.

For example, a program may contain separate functions for:

- Taking input
- Calculating total
- Calculating average
- Finding maximum value
- Displaying output

A function is a named, self-contained block of statements that performs a specific task and can be called from another part of the program.

Example of a Function

```
#include <stdio.h>

void message(void)
{
    printf("Welcome to C Programming\n");
}

int main(void)
{
    message();
    return 0;
}
```

Here, `message()` is a function. When:

```
message();
```

is executed, control transfers to the function and `"Welcome to C Programming"` is displayed.

Need and Importance of Functions

Functions provide several advantages in program development.

1. Modularity

A large program can be divided into smaller modules or functions.

2. Code Reusability

A function can be called multiple times, reducing repetition of code.

3. Easy Debugging

Errors can be identified and corrected within individual functions.

4. Easy Testing

Each function can be tested separately.

5. Better Readability

A program divided into meaningful functions is easier to understand.

6. Easy Maintenance

Changes can often be made to a particular function without unnecessarily modifying the entire program.

7. Reduced Program Size

Repeated operations can be placed in a function and reused whenever required.

Types of Functions in C

Functions can broadly be classified into **two types**:

1. Library Functions 2. User-Defined Functions

1. Library Functions

Library functions are functions provided by the C standard library for commonly required operations.

Examples include:

```
printf()
scanf()
strlen()
sqrt()
```

Different library functions are declared in appropriate header files.

For example:

```
#include <stdio.h>
```

provides declarations for standard input/output functions such as `printf()` and `scanf()`.

Similarly:

```
#include <string.h>
```

provides declarations for string-handling functions such as `strlen()`.

2. User-Defined Functions

A **user-defined function** is a function created by the programmer according to the requirements of the program.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

Here, `add()` is a user-defined function.

Library vs User-Defined Functions

Library Functions	User-Defined Functions
Provided by the standard library	Created by the programmer
Used for common operations	Used for specific program requirements
Declared through appropriate headers	Declared/defined by the programmer
Example: `printf()`	Example: `add()`

Three Important Aspects of a User-Defined Function

For exam purposes, remember that working with a user-defined function commonly involves:

Function Declaration → Function Call → Function Definition

For example:

```
#include <stdio.h>

/* Function Declaration */
int add(int, int);

int main(void)
{
    int result;

    /* Function Call */
    result = add(10, 20);

    printf("Sum = %d\n", result);

    return 0;
}

/* Function Definition */
int add(int a, int b)
{
    return a + b;
}
```

Output:

```
Sum = 30
```

Function Declaration

A **function declaration**, commonly called a **function prototype**, tells the compiler about a function's name, return type, and parameter types before the function is called.

General Syntax

```
return_type function_name(parameter_types);
```

Example:

```
int add(int, int);
```

or:

```
int add(int a, int b);
```

The parameter names are optional in a prototype.

Here:

`int` → Return type
`add` → Function name
`int, int` → Types of parameters

More Examples

```
void display(void);
float average(int, int);
int maximum(int, int);
```

Importance of Function Declaration

A function declaration allows the compiler to check whether a function is being called with appropriate argument types and whether its return value is being used consistently with its declared type.

A function declaration or prototype specifies the name, return type, and parameter types of a function before its use.

Function Call

A **function call** is an expression that invokes a function so that the statements in its body can be executed.

Example:

```
add(10, 20);
```

If the function returns a value, that value can be stored or used:

```
result = add(10, 20);
```

When the function is called, control transfers to the called function. After the function finishes, control normally returns to the calling function.

Flow: `main()` → **Function Call** → **Called Function** → **Return** → `main()`

Defining a Function

A **function definition** contains the actual implementation or body of a function.

General Syntax

```
return_type function_name(parameter_list)
{
    declarations;
    statements;
    return value;
}
```

Example:

```
int add(int a, int b)
{
    int sum;

    sum = a + b;

    return sum;
}
```

Parts of a Function Definition

A function definition mainly contains:

1. Return Type

Specifies the type of value returned by the function.

```
int
```

2. Function Name

The name through which the function is identified and called.

```
add
```

3. Parameter List

Contains the parameters that receive values supplied by the caller.

```
(int a, int b)
```

4. Function Body

The statements enclosed within `{ }` constitute the function body.

```
{
    int sum;
    sum = a + b;
    return sum;
}
```

5. return Statement

The `return` statement ends the function call and can return a value to the calling function.

```
return sum;
```

Classification of User-Defined Functions

This is one of the **most important examination topics**.

Functions can be classified according to whether they:

- Receive arguments
- Return a value

On this basis, there are four common categories:

Category	Arguments	Return Value
1. No arguments, no return value	No	No
2. Arguments, no return value	Yes	No
3. No arguments, returns value	No	Yes
4. Arguments, returns value	Yes	Yes

1. No Arguments and No Return Value

This type of function neither receives arguments from the calling function nor returns a value.

Example

```
#include <stdio.h>

void add(void)
{
    int a = 10, b = 20;

    printf("Sum = %d\n", a + b);
}

int main(void)
{
    add();
    return 0;
}
```

Output:

```
Sum = 30
```

Here:

Arguments = None
Return value = None

2. Arguments but No Return Value

The function receives values through arguments but does not return a value.

Example

```
#include <stdio.h>
```

```
void add(int a, int b)
{
    printf("Sum = %d\n", a + b);
}

int main(void)
{
    add(10, 20);
    return 0;
}
```

Here, `10` and `20` are supplied by the calling function.

The function receives them in:

```
int a, int b
```

but does not return a value.

3. No Arguments but Returns a Value

This type of function does not receive arguments but returns a value to the calling function.

Example

```
#include <stdio.h>

int add(void)
{
    int a = 10, b = 20;

    return a + b;
}

int main(void)
{
    int result;

    result = add();

    printf("Sum = %d\n", result);

    return 0;
}
```

Here:

Arguments = None

but the function returns:

```
return a + b;
```

4. Arguments and Return Value

This type of function receives arguments and also returns a result.

Example

```
#include <stdio.h>
```

```
int add(int a, int b)
{
    return a + b;
}

int main(void)
{
    int result;
    result = add(10, 20);
    printf("Sum = %d\n", result);
    return 0;
}
```

Output:

```
Sum = 30
```

Here:

Arguments = `10, 20`

and:

Returned value = `30`

This category is widely used because input values can be supplied by the caller and the calculated result can be returned for further processing.

Arguments and Parameters

These two terms should not be confused.

Arguments

The values or expressions supplied in a function call are called **arguments**.

Example: `add(10, 20);`

Here, `10` and `20` are arguments.

Parameters

The variables declared in the function definition that receive corresponding argument values are called **parameters**.

```
int add(int a, int b)
```

Here, `a` and `b` are parameters.

Difference between Arguments & Parameters

Arguments	Parameters
Supplied in function call	Declared in function definition/declaration
Provide values to the function	Receive/use corresponding values
Example: `add(10,20)`	Example: `add(int a, int b)`

Return Value of a Function

A function can return a value to its caller by using the `return` statement.

Example:

```
return sum;
```

Consider:

```
int square(int n)
{
    return n * n;
}
```

Calling:

```
result = square(5);
```

gives:

```
result = 25
```

A function declared with `void` does not return a value:

```
void message(void)
{
    printf("Hello\n");
}
```

Complete Example of User-Defined Function

```
#include <stdio.h>

/* Function Declaration */
int maximum(int, int);

int main(void)
{
    int a, b, result;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    /* Function Call */
    result = maximum(a, b);

    printf("Maximum = %d\n", result);

    return 0;
}

/* Function Definition */
int maximum(int x, int y)
{
    if (x > y)
        return x;
    else
        return y;
}
```

If input is:

```
20 35
```

Output:

```
Maximum = 35
```

In this program:

Declaration: `int maximum(int, int);`

Call: `maximum(a, b)`

Definition: `int maximum(int x, int y) { ... }`

Arguments: `a, b`

Parameters: `x, y`

Return type: `int`

Advantages of Functions

Functions are important because they provide:

Modularity – Large programs can be divided into smaller units.

Reusability – The same function can be called many times.

Readability – Program logic becomes easier to understand.

Easy Debugging – Errors can be located function by function.

Easy Maintenance – Individual functions can be modified more conveniently.

Reduced Duplication – The same code need not be written repeatedly.

Team Development – Different functions/modules can be developed by different programmers.

Passing Arguments to a Function

When a function is called, data can be supplied to it so that the function can perform operations on that data. The values or expressions supplied to a function are called **arguments**.

The variables in the function definition that receive these values are called **parameters** or **formal parameters**.

Example

```
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}
```

```
int main(void)
{
    int result;

    result = add(10, 20);

    printf("Sum = %d\n", result);

    return 0;
}
```

Here:

10 and 20 → Arguments
a and b → Parameters

The values 10 and 20 are passed to the function add().

Actual Arguments and Formal Parameters

Arguments are often explained using two terms:

1. Actual Arguments

The values, variables, or expressions supplied in a function call are called **actual arguments**.

add(x, y);

Here, x and y are actual arguments.

2. Formal Parameters

The parameters declared in the function definition are called **formal parameters**.

int add (int a, int b)

Here, a and b are formal parameters.

Actual Arguments	Formal Parameters
Appear in function call	Appear in function declaration/definition
Supply values	Receive corresponding values
Example: add(x, y)	Example: add(int a, int b)

Call by Value

In **call by value**, the value of an argument is copied into the corresponding function parameter.

Changes made to the parameter inside the called function do **not** change the original variable in the calling function.

Definition: Call by value is a method in which copies of argument values are passed to the parameters of a function.

C uses **pass-by-value** for function arguments.

Example

```
#include <stdio.h>

void change(int x)
{
    x = 100;
    printf("Inside function: %d\n", x);
}

int main(void)
{
    int a = 10;

    change(a);

    printf("Inside main: %d\n", a);

    return 0;
}
```

Output:

Inside function: 100
Inside main: 10

Advantages

- Original variables are protected from direct modification through the parameter.
- It is simple to understand and use.
- It reduces unintended changes to caller variables.

Call by Reference / Passing Addresses

Many introductory textbooks describe **call by reference** as passing the address of a variable so that a function can modify the original value.

In C, the precise explanation is:

C passes all arguments by value. To obtain reference-like behavior, the address of a variable is passed by value to a pointer parameter.

Thus, pointers allow a function to access and modify the caller's original object.

Example

```
#include <stdio.h>

void change(int *x)
{
    *x = 100;
}

int main(void)
{
    int a = 10;
    change(&a);
    printf("Value of a = %d\n", a);
    return 0;
}
```

Output:

Value of a = 100

Call by Value vs Call by Reference Style

This is a very important examination question.

Call by Value	Address/Reference Style
Value of variable is passed	Address of variable is passed
Function parameter receives a copy of the value	Pointer parameter receives a copy of the address
Direct changes to parameter do not affect original variable	Modification through the pointer can affect original variable
Ordinary parameters are used	Pointer parameters are used
Example: fun(a)	Example: fun(&a)

Recursive Function

A function that calls itself, directly or indirectly, is called a **recursive function**, and the process is known as **recursion**.

Definition: Recursion is a programming technique in which a function calls itself to solve a problem by reducing it into smaller instances.

A recursive function normally requires:

1. **Base case**
2. **Recursive case**

The base case stops further recursive calls.

Example of Recursive Function – Factorial

The factorial of a positive integer is:

5! = 5 × 4 × 3 × 2 × 1 = 120

Mathematically:

n! = n × (n − 1)!

with:

0! = 1

C Program

```
#include <stdio.h>

long long factorial(int n)
{
    if (n <= 1)
        return 1;

    return n * factorial(n - 1);
```

```
}

int main(void)
{
    int n = 5;
    printf("Factorial = %lld\n", factorial(n));
    return 0;
}
```

Output:

Factorial = 120

Working

For factorial(5):

factorial(5)
= 5 × factorial(4)
= 5 × 4 × factorial(3)
= 5 × 4 × 3 × factorial(2)
= 5 × 4 × 3 × 2 × factorial(1)
= 5 × 4 × 3 × 2 × 1
= 120

When n <= 1, recursion stops. This is the **base case**.

Advantages

- Makes some problems easier to express.
- Can reduce the amount of code for naturally recursive problems.
- Useful in tree processing and divide-and-conquer algorithms.
- Can provide an elegant mathematical representation.

Limitations

- Every active recursive call consumes stack space.
- Deep recursion may cause stack overflow.
- Function-call overhead may make recursion slower than an iterative solution in some cases.
- Incorrect or missing base cases can result in uncontrolled recursion.

Recursion vs Iteration

Recursion	Iteration
Uses repeated function calls	Uses loops
Requires a base case	Requires a loop termination condition
Uses call-stack space	Usually uses less call-stack space
Can be elegant for recursive problems	Often efficient for simple repetition

Writing a Multifunction Program

A **multifunction program** is a program divided into several functions, with each function performing a particular task.

Instead of placing all statements inside main(), the program is organized into separate functions.

For example:

main() → add() → subtract() → multiply()

This approach supports **modular programming**.

Example of Multifunction Program

```
#include <stdio.h>

int add(int, int);
int subtract(int, int);
int multiply(int, int);

int main(void)
{
    int a = 20, b = 10;

    printf("Addition = %d\n", add(a, b));
    printf("Subtraction = %d\n", subtract(a, b));
    printf("Multiplication = %d\n", multiply(a, b));

    return 0;
}

int add(int x, int y)
{
    return x + y;
}

int subtract(int x, int y)
{
    return x - y;
}

int multiply(int x, int y)
{
    return x * y;
}
```

Output:

Addition = 30
Subtraction = 10
Multiplication = 200

Advantages of Multifunction Programs

- Program becomes modular.
- Each function performs a specific task.
- Testing and debugging become easier.
- Functions can be reused.
- Large programs become easier to understand.
- Maintenance becomes easier.

Function Prototypes

A **function prototype** is a function declaration that specifies the function's name, return type, and parameter types before the function is called.

Syntax:

```
return_type function_name(parameter_types);
```

Example:

```
int add(int, int);
```

or:

```
int add(int a, int b);
```

Both declare a function named add that takes two int arguments and returns an int.

Examples of Function Prototypes

```
void display(void);
int sum(int, int);
float average(float, float);
char grade(int);
```

Importance of Function Prototypes

Function prototypes are important because they:

1. Inform the compiler about a function before its use.
2. Specify the function's return type.
3. Specify the number and types of parameters.
4. Enable compile-time checking of function calls.
5. Help detect mismatched argument types.
6. Improve program organization and readability.